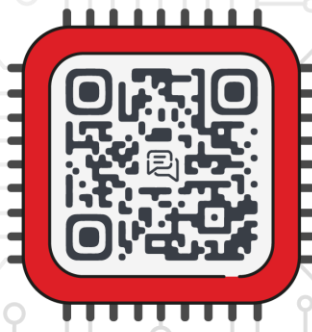




نظري

4

18

2700 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Testing Process

د. سعيد أبو طراب



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 06/05/2026

هندسة برمجيات ٢

Testability (قابلية الاختبار):

بدايةً وقبل القيام بعملية الاختبار يجب أن نعلم أولاً ما إذا كان النظام قابلاً للاختبار من الأساس، ويعود ذلك لنوع المتطلبات، فهناك متطلبات غامضة وغير واضحة لا يمكن اختبارها.

مثل:

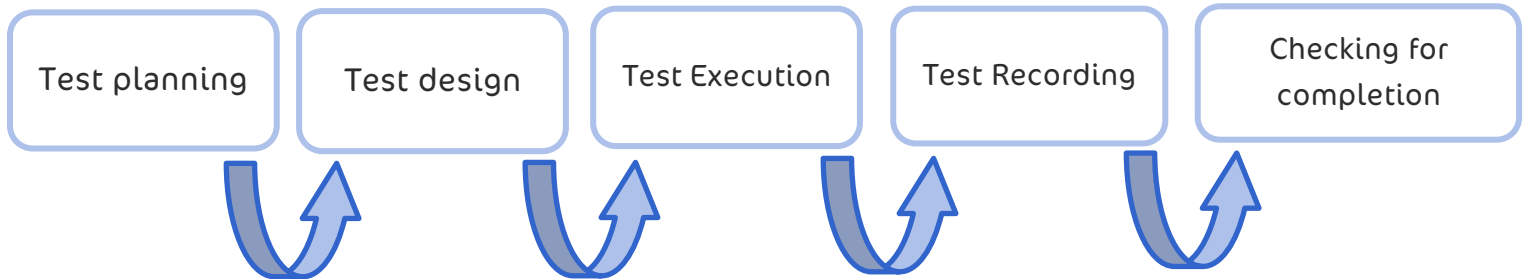
- يجب على النظام أن يكون سريعاً
- يجب على النظام أن يكون سهل الاستخدام
- يجب على النظام أن يكون فعالاً

مثل هذه المتطلبات لا يمكن اختبارها، وبالتالي يجب تجنبها واستبدالها بمتطلبات قابلة للقياس (Measurable) والاختبار كأن نقول يجب على النظام ألا تتجاوز مدة استجابته 2 ثانية بدلاً من القول يجب على النظام أن يكون سريعاً

فبمجرد التفكير كيف يمكن أن أختبر هذه الميزة؟ ممكن أن نكتشف نقص بالمتطلبات أو غموض أو تناقض.

عملية الاختبار (Test Process)

عند القول Testing Engineering فنحن فعلياً نقوم ب: Process، أي فهي ليست عملية عشوائية بل سلسلة مراحل مترابطة وهي:



متى نبدأ عملية الاختبار؟

ذكرنا ان عملية الاختبار تحدث على مرحلتين:

2. Execution التنفيذ

1. Test cases generation توليد حالات الاختبار

إنّ تحديد حالات الاختبار (inputs + expected outputs) يحدث في مرحلة ال design أي قبل وجود الكود
أما التنفيذ فيحدث بعد وجود الكود

وبالتالي الاختبار لا يبدأ من الكود بل يبدأ من المتطلبات

↪ مراحل ال Test Process :

1. Test Planning (تخطيط الاختبار):

يتم في هذه المرحلة تحديد الهدف بعيد المدى من الاختبار ووضع خطة له، مثلاً على أي مستوى نريد اختبار النظام؟
هل الهدف اختبار كونه قوياً وقابلاً للتوسع؟ أم الهدف فقط التأكد من أن الميزات Features صحيحة؟
ويتم تحديد ذلك بناءً على نوع التطبيق ومستوى خطورته فتشمل الخطة تحديد :

1. الوقت time : متى سيقوم الفريق بإنجاز الاختبار وإنشاء جدول زمني لذلك "schedule".
 2. التكلفة cost : كلفة عملية الاختبار حسب الاستراتيجية والموارد المتاحة والمخاطر.
 3. Strategy استراتيجية الاختبار:
- قد تكون معتمدة على الشركة المطوّرة، أي استراتيجية ثابتة تعتمد عليها الشركة لجميع المنتجات.
 - أو على المنتج بحيث كل منتج يخضع لاستراتيجية مختلفة حسب طبيعته.

فمثلاً إذا كان لدينا نظام تحكم بالطيران وبرنامج كتابة (Word) هل يتم اختبارهم بنفس الطريقة؟

بالتأكيد لا، الاستراتيجية ذاتها لا تجدي نفعاً لكلا النظامين لأن كل نظام له Risk مختلف وبالتالي له طريقة اختبار مختلفة، فالمنتجات التي قد يؤدي فشلها إلى خسائر مادية أو بشرية (مثل نظام مراقبة الملاحة الجوية)، يجب أن نقوم باختبارها اختباراً متيناً (Robust) ولن نقبل بظهور أخطاء أبداً مهما كان حجمها.

ماذا نحدد ضمن الاستراتيجية:

1. أنواع الاختبارات (Testing Types) مثل: integration testing, unit testing
2. طرق الاختبارات مثل black box, white box (سيتم شرحها لاحقاً)
3. الأدوات والبيئة المناسبة للاختبار
4. الأمور التي سنركز عليها في الاختبار مثلاً (إذا كان للنظام عدد من الوظائف، قد يكون بعضها أكثر أهمية من الآخر).
5. فريق الاختبار team
6. Policy: سياسة الاختبار تحدد الأهداف والمسؤوليات والمعايير المتعلقة بعملية الاختبار وتعتمد بشكل أساسي على كم نسمح بظهور أخطاء ضمن التطبيق، ليس فقط كعدد بل أيضاً كتصنيف حيث نصنف الأخطاء إلى: major faults , minor faults

ففي بعض الأحيان يسمح بوجود الأخطاء الصغيرة و لكن يجب اكتشاف الكبيرة منها لذا نحاول توليد اختبارات تهدف إلى اكتشافها.

2. Test Design (تصميم الاختبار):

هي مرحلة توليد حالات الاختبار test case generation وذلك بناء على المعايير التي تم تحديدها ضمن عملية التخطيط planning

3. Test Execution (تنفيذ الاختبار):

هي مرحلة تنفيذ حالات الاختبار على النظام، وهنا نحتاج كود فعلي نطبق عليه الاختبار.

▪ نذكر هنا مفهوم مهم Automated Testing (الاختبار الآلي):

وهو تشغيل الاختبارات بشكل تلقائي بدون تدخل بشري باستخدام أدوات أو Frameworks مثل: Junit أو أي أدوات Testing Automation
هذه الأدوات تسمح لنا بكتابة Test Cases مرة واحدة وتشغيلها مئات المرات تلقائياً والحصول على تقرير بالنتائج
وهذا مهم لأنه أسرع بكثير من الاختبار اليدوي و أدق بحيث يتم تجنب الأخطاء البشرية بالمقارنة.
لكن مشكلته انه عند أي تغيير في سلوك النظام يجب تغيير script الاختبار وهي عملية مجهد.

4. Test Recording (تسجيل النتائج):

هي مرحلة توثيق نتيجة كل Test Case لتحديد حالات الفشل والنجاح بعد مقارنة الخرج المتوقع مع الخرج الفعلي للنظام
الهدف: تتبع الأخطاء والرجوع إليها لاحقاً وتحليل الأداء.

5. Checking for Completion

ويأتي السؤال الصعب: متى نوقف Testing ؟

الجواب الصحيح

عندما نحقق ال Test Criteria وهو: معيار يحدد إذا كانت مجموعة الاختبارات كافية أم لا



الجواب الخاطئ

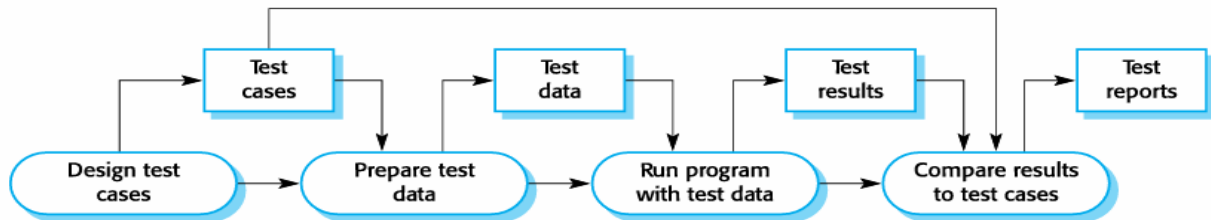
عندما ينتهي الوقت أو الميزانية



هذا المعيار ممكن أن يكون:

- نسبة تغطية الكود (Coverage) (سيتم شرحها لاحقاً)
- عدد الأخطاء المكتشفة

من المستحيل اختبار كل الحالات الممكنة لذلك نحدد مستوى كافٍ للاختبار بناءً على المخاطر، الكلفة، والوقت.



Test Phases

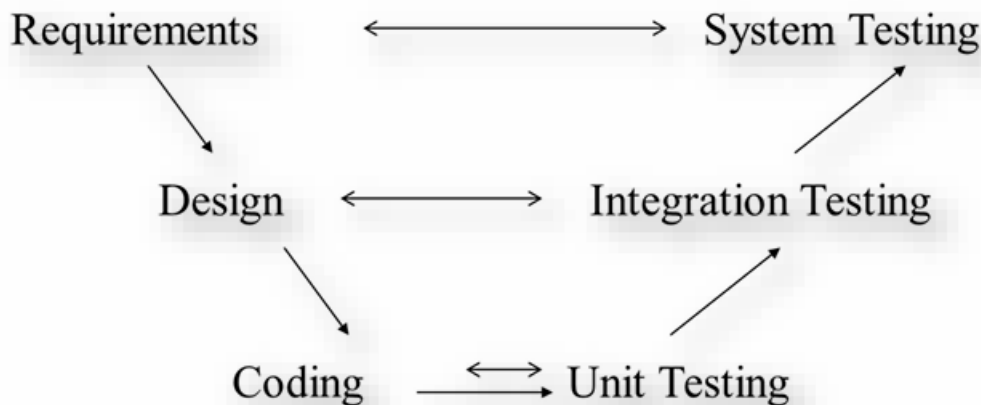
يتم تقسيم عملية اختبار النظام إلى ثلاث مراحل رئيسية:

1. Development Testing
2. Release Testing
3. User Testing

أولاً: Development Testing

هو الاختبار الذي يتم خلال مرحلة التطوير ويهدف لاكتشاف الأخطاء في وقت مبكر، يوجد لدينا عدة أنواع من الاختبارات التي تحدث ضمن هذه المرحلة لكل منها هدف محدد وارتباط مع مرحلة تطوير مختلفة للنظام. النموذج التالي يوضح هذه العلاقة بين مراحل التطوير (Development) ومراحل الاختبار (Testing) وهذا النموذج هو

V-Model

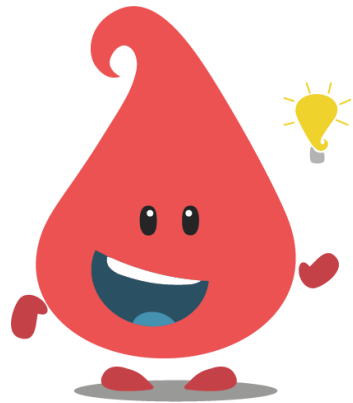


سُمِّي بـ V-Model لأنه يأخذ شكل حرف V

■ الجهة اليسرى (↓) تمثل: مراحل تطوير النظام

ملاحظة:

عند النزول في الجهة اليسرى نحن لا نقوم بالبناء فقط بل نقوم أيضاً بالتحقق من صحة كل مرحلة أي:
التأكد أن كل مرحلة (Deliverable) صحيحة ومتوافقة مع المرحلة السابقة وفي هذه المرحلة أيضاً يمكن إشراك المستخدم (User) للتأكد من أن المتطلبات مفهومة بشكل صحيح وما يتم بناؤه يحقق ما يريده فعلاً.
وخلال النزول في الجهة اليسرى، نقوم أيضاً ب: **تطوير خطة الاختبار (Test Plan)**



لماذا هذا مهم؟

لأنه يساعد في تقليل تضخم الأخطاء (Defect Multiplication)، فإذا تم اكتشاف الخطأ مبكراً لا ينتقل إلى المراحل التالية ولا يتضاعف تأثيره
فالأخطاء غير المكتشفة في المتطلبات تؤدي إلى أخطاء في التصميم.
والأخطاء غير المكتشفة في التصميم تؤدي إلى أخطاء في التنفيذ.
لذلك، تأتي الأخطاء في التنفيذ من:
✚ المتطلبات، التصميم، والبرمجة.

ملاحظة: validation & verification لا يبدأان بعد كتابة الكود، بل يجب أن يحدثا طوال عملية التطوير.

■ الجهة اليمنى (↑) تمثل: مراحل الاختبار

على ماذا تدل الأسهم التي تربط كل مرحلة بما يقابلها؟ أي ما الرابط بينهم؟

السهم: → يدل على مصدر توليد test cases

Requirement → system testing

أي أن مصدر حالات الاختبار التي سيتم توليدها لاختبارات ال system هي المتطلبات.

Design → integration testing

أي أن مصدر حالات الاختبار التي سيتم توليدها لاختبارات ال integration (وهي تفاعل المكونات مع بعضها) هي مخططات التصميم.

Code → unit testing

أي أن مصدر حالات الاختبار التي سيتم توليدها لاختبارات الوحدات هو وصف الكود.

السهم: ←

يدل على مكان الخطأ الواجب تصحيحه في حال وجوده ضمن اختبار معين.

Requirement ← system testing

أي في حال كان الخطأ في system test سيكون التعديل على مستوى المتطلبات.

Design ← integration testing

أما في حال كان الخطأ في integration test سيكون التعديل على مستوى ال design.

Code ← unit testing

أما عند وجود خطأ في unit test تصحيح الخطأ يكون على مستوى الكود.

نلاحظ أن: اكتشاف خطأ على مستوى system سيجعلنا نقوم بالتعديل والتصحيح على مستوى المتطلبات وبالتالي التعديل على ال Design ومن ثم ال Code الكود وهي سلسلة طويلة ومكلفة أما إذا اكتشفته في Unit Testing التعديل يكون سريع ومباشر.

أنواع الاختبار ضمن V-Model

1. Unit Testing

اختبار الوحدات هو عملية اختبار الوحدات الفردية بشكل منفصل وهذه الوحدات قد تكون:

- Function (تابع)
- Object (كائن) في البرمجة الكائنية (OOP)
- components مركبة ذات واجهات محددة تستخدم للوصول إلى وظائفها

يتم هذا الاختبار بواسطة المطور

مثال:

لدينا تابع:

```
int sum (int x, int y) {
    return x + y;
}
```

كيف نقوم باختباره؟

نحتاج إلى:

- إدخال قيم (Input)
- تنفيذ التابع
- مقارنة الناتج

وللتنفيذ نكتب كوداً إضافياً مثل:

```
int x = sum (3, 2);
cout << x;
```



هل هذا الكود جزء من النظام الحقيقي؟

لا، هذا يسمى Driver

■ وهو كود إضافي (Extra Code) ليس جزءاً من النظام الحقيقي تتم كتابته لغرض اختبار وحدة معينة من النظام، يقوم بتمرير القيم (Input) إلى الوحدة المراد اختبارها واستقبال النتائج (Output) منها وعرض أو تسجيل هذه النتائج.

لكن إذا أردنا تغيير test cases سنضطر لتعديل الكود كل مرة وإعادة التشغيل، لذلك نلجأ لحلول بديلة مثل:

1. إدخال البيانات من المستخدم: عبر الشاشة أو باستخدام Loop لكن ما يزال الأمر يدوياً
2. القراءة من ملف مثل Excel : قراءة القيم من خلايا وتنفيذ الاختبار تلقائياً وتسجيل النتائج
3. Automated Testing (الاختبار الآلي)

2. Integration Testing

في الـ unit test تأكدنا من كون كل وظيفة تعمل لوحدها، أما هنا سنقوم بالتأكد من صحة هذه الوظائف مع بعضها أي سنقوم باختبار تكامل الـ units فيما بينها بشكل صحيح، وتدعى استراتيجية البناء التزايدية، أي نقوم بفحص المستويات بشكل تزايدى لتسهيل الاختبار في تغطية كامل المنتج.

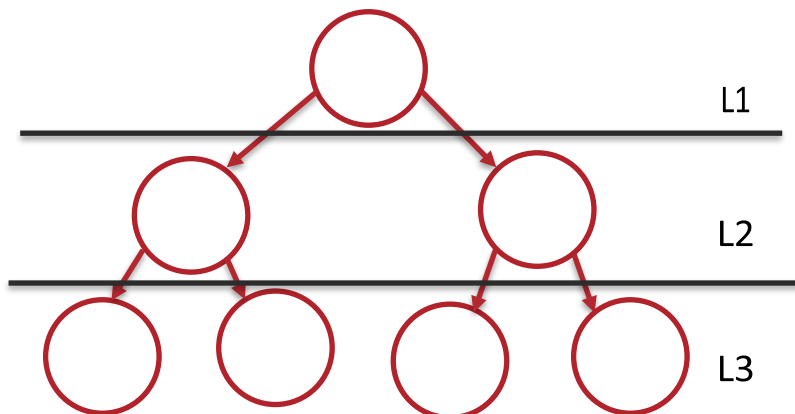
لنتخيل الحالة التالية لديك مئات الكائنات (Objects) كل كائن يتفاعل مع غيره عبر علاقات متعددة، والمشكلة عدد العلاقات كبير جداً (ثنائيات، ثلاثيات، ...)، لذلك يجب اختيار استراتيجية مناسبة للدمج (Integration Strategy)

فائدة:

- في البرمجة الإجرائية (Procedural) يتم استخدام: أسلوب الشجرة (Top-Down / Bottom-Up)
- في OOP يتم الاعتماد على Feature-Based أو Use Case-Based أي نختبر كل ميزة (Feature) أو سيناريو (Flow) بشكل مستقل.

سنقوم بتمثيل النظام بشكل شجري، لدينا root معينة ومن يتواصل معها بشكل مباشر يدعى المستوى الأول، ومن يُستدعى عن طريق المستوى الأول يدعى المستوى الثاني وهكذا...

نقوم بمعرفة الـ components التي تفاعلت مع بعضها على مستوى use case واحدة حيث يوجد لها مسار داخلي ضمن النظام يتم بتفاعل المستخدم مع النظام عن طريق الواجهة، ومن ثم يتم التفاعل ضمن سلسلة من مكونات النظام تنتهي بقاعدة البيانات وتنفيذ الوظيفة المطلوبة.

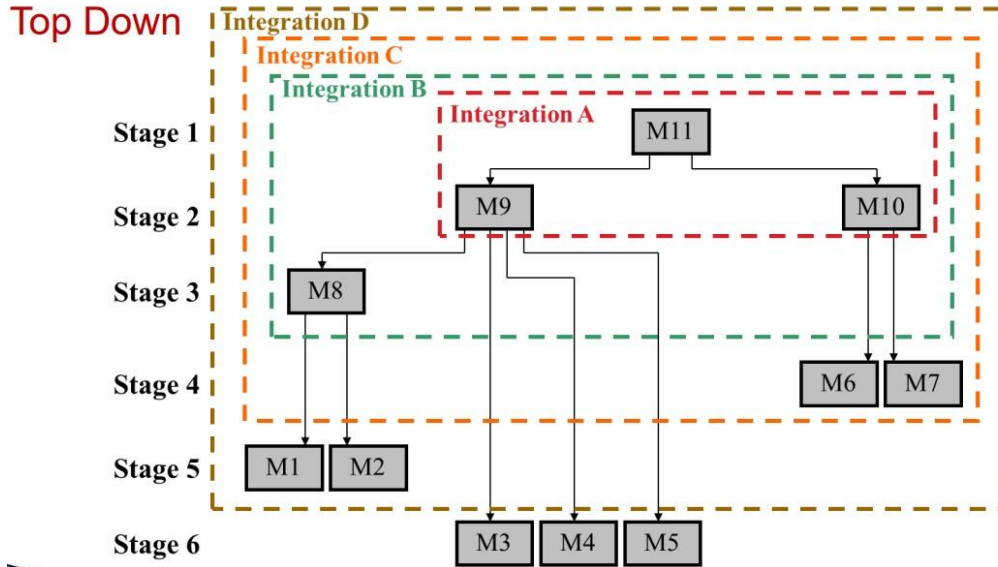


أنواع Integration Testing استراتيجيات التكامل:

عند تنفيذ اختبار التكامل (Integration Testing)، لا توجد طريقة واحدة ثابتة، بل توجد عدة استراتيجيات يمكن اتباعها، ويعتمد اختيارها على طبيعة النظام وبنية مكوناته.

1. التكامل من الأعلى إلى الأسفل (Top-Down Integration):

في هذه الاستراتيجية نبدأ باختبار الوحدات عالية المستوى (High-Level Modules) ثم نتقل تدريجياً نحو الوحدات الأدنى (Low-Level Modules) أي من الواجهات أو المكونات الرئيسية، ثم النزول إلى التفاصيل.



لكن هنا نواجه مشكلة وهي أننا بمستوى معين قد نحتاج لاستدعاء تابع من المستوى الذي في الأسفل غير الجاهز بعد، هنا نقوم باستخدام ما يدعى ب Stub وهو كود وهمي (Dummy Code) يُستخدم كبديل مؤقت لجزء من النظام غير الجاهز.

نستخدمه إذا كانت الوحدة التي نريد اختبارها قد تعتمد على مكونات أخرى غير مكتملة بعد أو معقدة أو نرغب في عزلها أثناء الاختبار، فمهمته تقليد سلوك المكون الحقيقي وإعادة نتائج بسيطة أو ثابتة.

مثال:

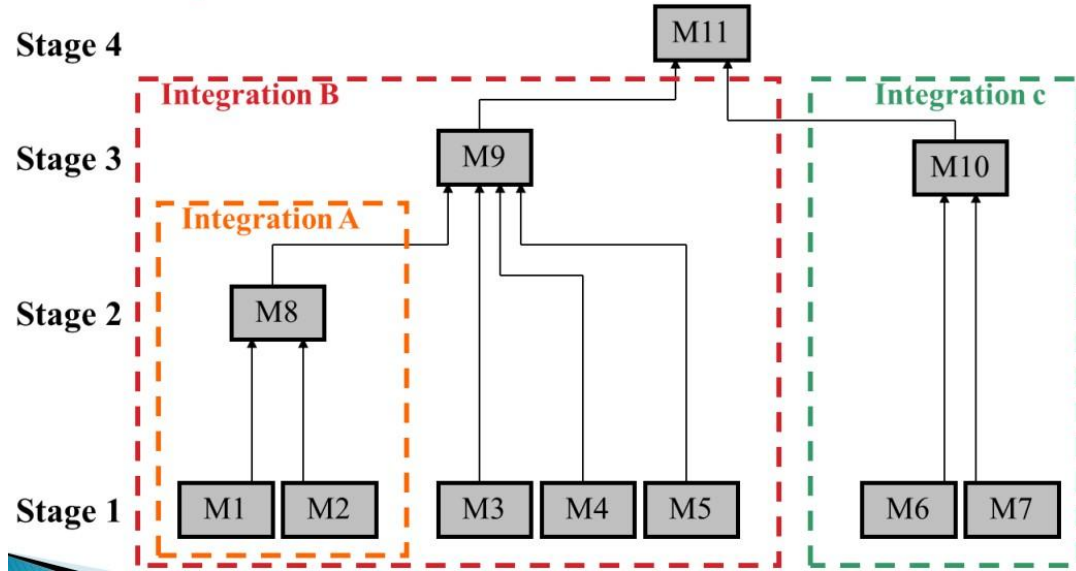
إذا كان لدينا تابع يعتمد على دالة لجلب بيانات من قاعدة بيانات، وهذه الدالة غير جاهزة بعد، يمكن استبدالها ب Stub يقوم بإرجاع قيمة ثابتة مثل: return 100; ولاحقاً نقوم باستبدال ال stub بالكود الحقيقي.

2. التكامل من الأسفل إلى الأعلى (Bottom-Up Integration):

في هذه الحالة نبدأ بالوحدات الصغيرة والبسيطة (Low-Level Module) ثم نقوم بدمجها تدريجياً لبناء المكونات الأعلى.



Bottom Up



وهنا أيضاً نواجه مشكلة وهي كيفية اختبار الوحدات السفلية قبل اكتمال النظام (ربما قبل وجود واجهات تساعدنا على إدخال المدخلات لإجراء الاختبار)، لذا نلجأ إلى الـ driver.

مقارنة بين Stub و Driver:

العنصر	الوظيفة	الاستخدام العملي
Driver	تشغيل الوحدة وتمرير المدخلات واستقبال النتائج	يستخدم عندما لا يوجد نظام كامل لتشغيل الوحدة
Stub	استبدال جزء غير جاهز من النظام	يستخدم عندما تكون بعض المكونات غير مكتملة

3. التكامل المختلط (Sandwich Integration):

تُعرف أيضاً بـ **Hybrid Integration** وهي الجمع بين Top-Down و Bottom-Up حيث يتم العمل من الأعلى نزولاً والعمل من الأسفل صعوداً في نفس الوقت والهدف تسريع عملية الاختبار، وتقليل الزمن اللازم لاكتشاف الأخطاء في مختلف مستويات النظام.

3. اختبار النظام System Testing

- هو full integration testing أي اختبار النظام كامل (End-to-End) عندما يتم دمج جميع المكونات (Components) والوحدات (Units) للتأكد من أن "المنطق البرمجي" يعمل ككتلة واحدة، ويُعتبر المرحلة الختامية ضمن اختبارات التطوير (Development Testing).
- يتم ضمن بيئة التطوير الداخلية (development environment).
- يتم من قبل فريق التطوير نفسه.

- هدفه الرئيسي **اكتشاف العيوب (Defect Testing)** ، فالمختبر هنا يبحث عن الأخطاء الناتجة عن التفاعلات غير المتوقعة بين الواجهات (Interfaces) ، والتأكد من مطابقة المتطلبات الوظيفية، أي هدفه عمل **verification** للنظام.
- المنهجية المستخدمة في الاختبار White-box testing (أي نقوم بفحص كتابة الكود نفسه)

❗ والان هل ينتهي الاختبار عند System Testing ؟

الجواب: لا، رغم أن System Testing يعني اختبار النظام بشكل كامل (End-to-End) ، إلا أن ذلك لا يكفي لضمان جاهزية النظام للاستخدام الفعلي، لأن النظام قد يكون صحيحاً من الناحية التقنية، لكنه لا يزال غير مناسب للاستخدام الحقيقي للمستخدم، لذلك سننتقل الى المرحلة الثانية من مراحل اختبار النظام وهي **Release Testing**.

ثانياً: Release Testing (اختبار ما قبل الاطلاق):

- هو الاختبار الذي يتم على نسخة كاملة من النظام قبل تسليمها للمستخدم النهائي، ويهدف لإثبات جاهزية المنتج للتعامل وإقناعه بأن النظام "جيد بما يكفي للاستخدام"، فهو يُصنف كنوع من اختبارات التحقق من الصلاحية **Validation**.
- يكون الاختبار ضمن بيئة التحضير (Staging Environment) ، هذه البيئة هي نسخة مطابقة للبيئة الحقيقية (Production) من حيث السيرفرات، وقواعد البيانات، والحزم البرمجية (Packages) ؛ لضمان أن النظام سيعمل عند العميل دون مشاكل تقنية.
- يتم من قبل فريق اختبار مستقل (Independent Testing Team) لم يشارك في البرمجة، لأن المختبر المستقل ينظر للنظام بعين مختلفة عن المطور ويبحث عن الأخطاء بشكل أعمق.
- يهدف للتأكد أن النظام يعمل بشكل صحيح، مستقر وجاهز للإطلاق، وملئم لاحتياجات المستخدم.
- المنهجية المستخدمة في الاختبار Black-box Testing (أي تهتم بالدخل والخرج دون النظر للكود)

ثالثاً: User Testing (اختبار المستخدم):

هو الاختبار الذي يقوم به المستخدمون الفعليون أو المحتملون للنظام في بيئتهم الخاصة.

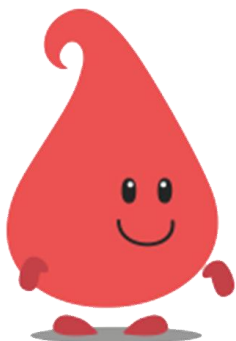
الهدف منه: التحقق من أن النظام يلبي احتياجات المستخدم ويعمل في البيئة الحقيقية (Real Environment) .

حتى لو تم إجراء اختبارات نظام وإصدار شاملة، يبقى اختبار المستخدم أساسياً لأن النظام قد يعمل بشكل ممتاز داخل الشركة، لكن عند استخدامه في بيئة المستخدم قد تظهر مشاكل لم تكن متوقعة.

يعود ذلك لعوامل بيئية وظروف واقعية لا يمكن محاكاتها بدقة في مختبرات الاختبار الخاصة بشركة التطوير.

أمثلة عن الاختلافات:

1. جهاز مختلف
2. نظام تشغيل مختلف
3. سرعة إنترنت مختلفة
4. طريقة استخدام مختلفة



أنواع User Testing

1. Alpha Testing :

يتم فيه دعوة مجموعة من المستخدمين للعمل مع فريق التطوير في موقع المطور (Developer's Site) ، هنا يقوم المستخدمون بتجربة النظام واكتشاف المشاكل التي قد لا يلاحظها المبرمجون بسبب تعودهم على الكود. (يكافئ ال release testing)

2. Beta Testing :

يتم من قبل مستخدمين حقيقيين في بيئتهم الخاصة، يهدف هذا الاختبار إلى اكتشاف العيوب الناتجة عن التفاعل بين النظام والظروف المتنوعة في بيئة التشغيل الفعلية، كما يُعد شكلاً من أشكال التسويق للمنتج.

Process Improvement (تحسين عملية الاختبار):

هناك نقطة مهمة وهي أن عملية الاختبار نفسها يجب أن تتطور مع الزمن، والاختبار (Testing) ليس عملية ثابتة نطبقها في كل مشروع بنفس الطريقة، بل هو عملية ديناميكية تحتاج إلى مراجعة وتحسين مستمر وذلك بالقيام بشكل دوري بـ:

- مراقبة نتائج الاختبارات السابقة
- تحليل الأخطاء التي تم اكتشافها
- تقييم فعالية أساليب الاختبار المستخدمة

لتحسين عملية الاختبار، يجب أن نطرح على أنفسنا سؤالين أساسيين:

ما هي أكثر أنواع الأخطاء التي تتكرر باستمرار؟

ما هي أنواع الأخطاء التي لم ننجح في اكتشافها أثناء الاختبار؟

■ الحالة الأولى: أخطاء تتكرر باستمرار

إذا لاحظنا أن نوعاً معيناً من الأخطاء يتكرر بشكل متكرر، فهذا يدل على أن هناك ضعف في استراتيجية الاختبار في هذا الجانب

■ الحالة الثانية: أخطاء تظهر عند المستخدم

إذا ظهرت أخطاء بعد تسليم النظام للمستخدم، ولم يتم اكتشافها أثناء الاختبار، فهذا يعني أن عملية الاختبار نفسها تحتاج إلى إعادة تقييم وتحسين.

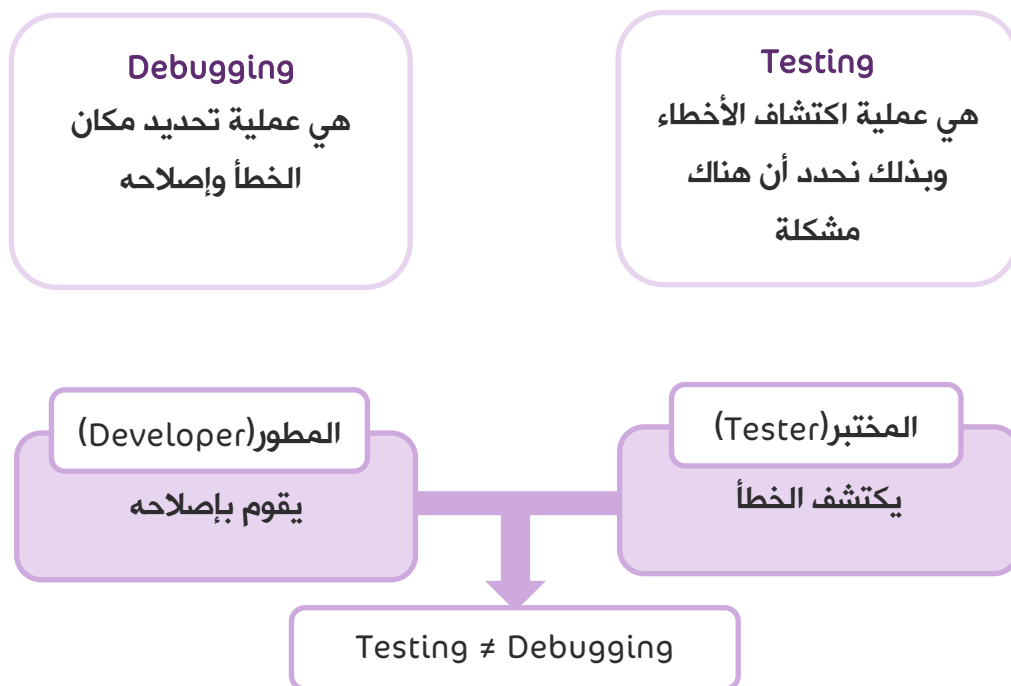
في الواقع، لا يمكن بشكل دائم اختبار النظام مباشرة في بيئته الحقيقية وفي بعض الحالات يكون ذلك خطيراً أو ليس عملياً

مثال: نظام للتحكم بالطيران، ليس من المنطق تطبيق أول اختبار ضمن طائرة حقيقية، لذلك نلجأ إلى ما نسميه بال **Test Environment** بيئة الاختبار، وهي بيئة تحاكي الواقع قدر الإمكان تهدف الى اختبار النظام بطريقة قريبة من الاستخدام الحقيقي بدون مخاطرة.

ممکن أن نستخدم:

- برامج محاكاة (Simulation)
- بيئات وهمية (Mock Environment)
- أنظمة مشاهدة للبيئة الفعلية

الفرق بين Debugging و Testing



حيث يقوم المختبر بذكر حالة الاختبار والدخل الذي ظهر عنده الخطأ ومن ثم يقوم المطور بعملية تحليل وتتبع tracing للكود لاكتشاف موقع الخطأ، وهنا تظهر خبرة المطور لأنه قد يكون لدينا مجموعة كبيرة من ال modules تتخاطب مع بعضها وعلينا اكتشاف في أي module يقع الخطأ تحديداً

إن عملية تصحيح الخطأ debugging تتم على مرحلتين:

1. اكتشاف موقع الخطأ (الامر الذي يأخذ الجهد أكبر)
2. تصحيح الخطأ

وعند تصحيح الخطأ في مكان ما قد يكون له أثر رجعي بمكان آخر، لذلك بعد عملية ال debugging يجب عمل ما يسمى ب Regression Testing.

Regression Testing: هو إعادة تنفيذ جميع الاختبارات السابقة والهدف التأكد من أن التعديلات الجديدة لم تُحدث أخطاء جديدة (Side Effects) أي عندما نكتشف خطأ ونقوم بإصلاحه هذا التعديل ممكن يسبب مشاكل في أماكن ثانية لذلك نستخدم Regression Testing .

يمكن إجراء هذا الاختبار يدوياً أو باستخدام الاختبار الآلي Automation.

فائدة: كلما كان النظام أكبر أصبح ال Automation ضرورة وليس خيار

التطوير المقاد بالاختبار (Test-Driven Development - TDD)

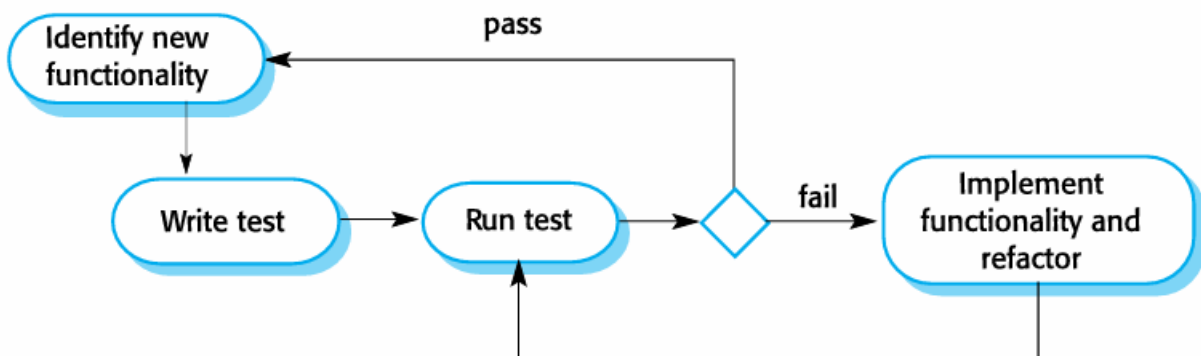
هي منهجية متطورة في هندسة البرمجيات تعتمد على تداخل عمليتي البرمجة والاختبار بشكل وثيق وتزايدي. حيث تقوم هذه الطريقة بعكس الطريقة التقليدية في التطوير (التنفيذ أولاً ثم الاختبار) ليصبح الاختبار هو المحرك الأساسي لعملية التطوير.

- يتم ذلك بكتابة الاختبارات قبل الكود، وبعدها "نجاح" الاختبارات هو العامل الأساسي لتطوير الكود.
- نقوم بتطوير الكود بشكل تدريجي incrementally، مع كتابة اختبار لكل جزء جديد increment، ولا ننتقل إلى الجزء التالي next increment إلا بعد أن ينجح الكود الذي قمنا بتطويره في الاختبار.
- يتم اعتماد TDD في نماذج التطوير التابعة لل agile مثل البرمجة القصوى (Extreme Programming)، لكن يمكن استخدامه أيضاً في نماذج التطوير التابعة لل plan-driven.

TDD Process

تتم العملية وفق دورة متكررة ومنظمة تتكون من خمس خطوات أساسية:

1. تحديد الميزة المراد تطويرها: من الأفضل أن تكون صغيرة ويمكن تنفيذها في بضعة أسطر من الكود.
 2. كتابة الاختبار: كتابة اختبار لهذه الميزة وتنفيذه كاختبار آلي (Automated Test)
 3. تشغيل الاختبار: في البداية، لم نقم بتنفيذ الوظائف، لذا سيفشل الاختبار الجديد.
 4. كتابة الكود التنفيذي: تنفيذ الوظائف وإعادة تشغيل الاختبار.
- بمجرد نجاح الاختبار، يتم الانتقال للميزة التالية



فوائد منهجية TDD :

تقدم هذه المنهجية مزايا استراتيجية تتجاوز مجرد كشف الأخطاء:

- تغطية الكود (Code Coverage) : تضمن هذه الطريقة أن كل سطر برمجي يتم كتابته يمتلك اختباراً واحداً على الأقل مرتبطاً به، مما يزيد الثقة في جودة النظام
- تكرار الاختبار (Regression Testing): يتم بناء مجموعة اختبارات متكاملة بشكل تزايدى incrementally ، ويمكن تشغيلها باستمرار للتأكد من أن التعديلات الجديدة لم تكسر الوظائف القديمة .
- تبسيط عملية حل الأخطاء (Simplified Debugging) : عندما يفشل اختبار ما، يكون من الواضح جداً أين تكمن المشكلة (في آخر كود تم كتابته).
- توثيق النظام: تعمل الاختبارات نفسها كنوع من التوثيق الحي الذي يشرح ما يجب أن يفعله الكود فعلياً
- فهم أفضل للمتطلبات: تجبر المبرمج على توضيح أفكاره وفهم ما يجب أن يقوم به الكود بدقة قبل البدء في كتابته.

منظورات اختبار البرمجيات (Testing Perspectives)

عندما نقوم بفحص أي شيء، إما نقوم بفحصه مع معاينة بنيته الداخلية أو نقوم بفحصه بناءً على وظائفه فقط. فمثلاً لاختبار عمل جهاز الإسقاط (البروجكتر) وظيفياً نقوم بوصله مع الشاشة ونضغط زر تشغيل ليعمل، ويمكن أن نقوم بتغيير أبعاد الشاشة من خلاله وكل هذا تم بدون معرفة من كم دارة يتكون وما هي مكوناته الداخلية، ومن هنا ظهر لدينا هذان المنظوران للاختبار:

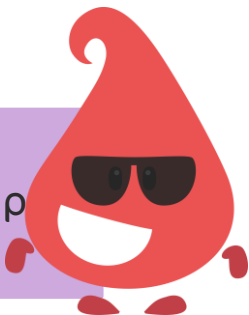
- **Black Box Testing**: فحص البرمجيات وظيفياً من وجهة نظر المستخدم، أي بدون رؤية البنية structure أو المكونات الخاصة بال Code حيث يعتمد تقييم الأداء على مدخلات البرنامج ونتائج الإخراج.
- **White Box Testing**: يكون عكس اختبار الصندوق الأسود حيث يتم اختبار البرمجيات بشكل مفصل و دقيق من خلال دراسة ال Code الخاص بالبرنامج.

سنقوم بالتركيز على white box testing وبالأخص على مستوى ال Unit test وذلك لأن المشكلة الأساسية تكمن في اختيار ال input المناسبة لاكتشاف الأخطاء.

ك Testing engineer يهمننا :

1. تحديد test cases مناسبة تؤدي إلى اكتشاف الأخطاء.
2. بناء Testing strategy/policy: يجب أن يكون لدينا معيار يحدد لماذا قمنا باختيار هذا ال test و متى علينا التوقف عن عملية الاختبار (لأن عملية الاختبار بطبيعتها لا تنتهي).

Don't give up



مثال: كود لحل معادلة من الدرجة الثانية:

```

1. double a, b, c, d, x1, x2;
2. d = b*b -4(a*c);
3. if (d >= 0) {
4.   x1 = (-b + sqrt(d)) / 2*a;
5.   x2 = (-b - sqrt(d)) / 2*a;
6.   cout<<x1<<x2};
7. else
8.   cout <<"no solution";
9. return 0;

```

المدخلات الخاصة بهذا التابع هي a, b, c والمخرجات هي x1, x2 .
 نولد حالات اختبار مختلفة من الشكل: (1, 1, 1) (1, 1, 1) (1, 1, 1) (1, 1, 1) (1, 1, 1) ...
 (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) (2, 2, 2) ...

بناء على هذه الاختبارات العشوائية هل نكون قد قمنا بفحص الـ Code بشكل صحيح؟!

فعلياً لا، لأننا بجميع الحالات قمنا بالفحص بدون معرفة الحالة البرمجية التي تؤدي الى هذه النتيجة.
 إذا نظرنا إلى الـ Code وتتبعنا مسار حالات الاختبار سنجد أننا قمنا بفحص نفس المسار لجميعها و التي تؤدي إلى قيمة دلتا سالبة، وباقي المسارات التي تؤدي إلى دلتا أكبر أو تساوي الصفر لم نقوم بفحصها.
 لذا تبين لدينا أنه رغم تطبيقنا لمجموعة من الـ test cases لم نقوم بفحص كامل الـ Code، بل قمنا بفحص segment معين منه فقط.

نتيجة: لمعرفة اختيار الـ test cases المناسبة والتي تعطي معامل تغطية محدد يجب علينا أن نكون قادرين على تخيل الـ Code وطريقة سيره.

بملاحظة الـ Code يجب علينا تتبع مسارات كل دخل و لكن هذا سيتطلب الكثير من الوقت و الجهد خاصة في الـ Codes الكبيرة لذلك الأسهل أن نمثل الـ Code ك flow و ذلك باستخدام CFG.

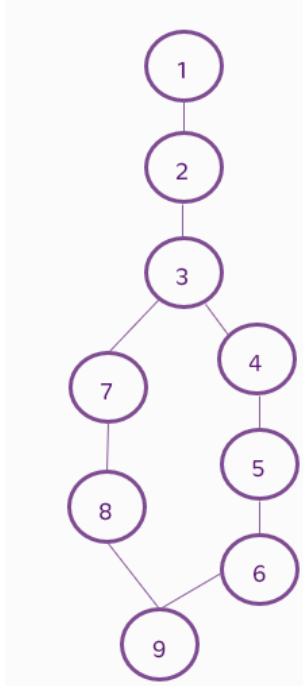
CFG: control flow graph

هنا سندمج الفكرة التصميمية والتنفيذية، حيث كل سطر ضمن الـ Code نقوم بتمثيله ب node أيما كان ما يحتويه حيث ما يهمنا هو الـ flow الخاص بالـ Code وليس كل statement وما تمثله.

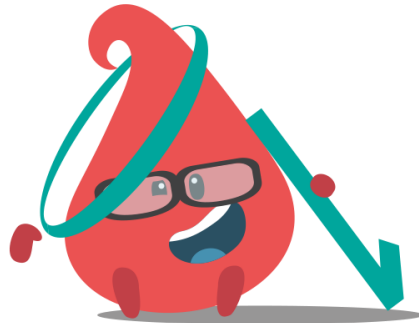
عندما تكون الـ node خارج منها فرعين "2 branches" فهي predicate:

- أول نقطة تسمى input case
- آخر نقطة تسمى output case

نمثل الـ Code السابق كـ CFG بحيث نعطي لكل node رقم السطر فيكون الشكل:



نلاحظ أن جميع الـ test cases اتبعت المسار (123789).
ومنه أصبح لدينا رؤية أفضل لما قمنا باختباره وما لم نختبره



معامل التغطية Coverage Criteria

نختار معاملات التغطية للـ Testing ضمن الـ policy حسب الـ application.

حتى نقول أن هذه التغطية كافية يجب أن نراعي:

- في حال كان التطبيق احتمالية فشله كبيرة نختار معامل تغطية أدق.
- أما في حال كان تطبيق سهل فنختار معامل تغطية أقل دقة

- إذا أردنا إجراء اختبارات أكثر علينا اختيار معامل تغطية أدق.

يوجد 3 أنواع لمعاملات التغطية:

1. Statement Coverage
2. Branch Coverage
3. Path Coverage

■ Path: هو تسلسل الانتقال من نقطة دخل إلى نقطة الخرج.

■ Test case: هي عبارة عن دخل input و الخرج المتوقع expected output و التي تقابل على الغراف Path نسميه testing Path

من هو معامل التغطية الأدق أو الذي يولد اختبارات أكثر؟

نلاحظ أنه لا يمكننا فحص كل statement على حدى أو خلق test case لها لأن جميع الـ statements لها دخل و خرج

للإجابة على السؤال سنجيب عن الأسئلة التالية بناء على المثال والمخطط التالي:



كم testing path نحتاج لنحصل على statement coverage ؟

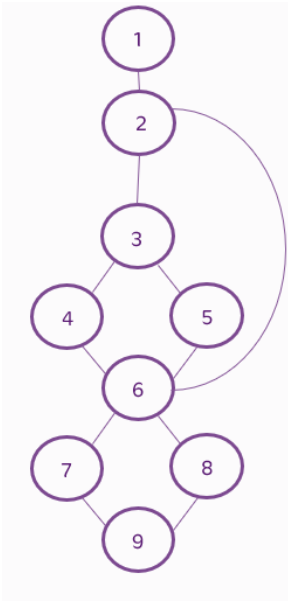
نحتاج إلى مسارين لتغطي العقد التسعة: (1234689) و (1235679)

كم testing path نحتاج لنحصل على branch coverage ؟

نحتاج إلى 3 مسارات لتغطي الأفرع الثلاثة: 12689 بالإضافة إلى مسارات الـ statement coverage.

كم testing path نحتاج لنحصل على path coverage ؟

نحتاج إلى 6 مسارات لتغطيتها جميعها وهي:
(1234689) و (1234679) و (1235679) و (1235689) و (12679) و (12689).



- معامل تغطية الـ branch يعطي اختبارات أكثر أي فهو أدق من الـ statement، وعندما نحقق الـ branch coverage نكون قد حققنا ضمناً الـ statement coverage.
- معامل تغطية الـ Path هو الأدق ويحقق ضمناً الـ branch + statement، وبعدها نقوم بتحديد الـ test cases اللازمة للمرور بكل الـ path.
- قد نصل إلى الـ infeasible path وهو مسار غير قابل للتحقق، ونقوم دائماً بالتحقق من وجوده لاستثناءه من عملية الاختبار.

مثال آخر: ليكن لدينا الـ code التالي، والمطلوب:

```
1. input (x, y);
2. z=0;
3. while (x+y>0) {
4.     if (x>0)
5.         x=x-1;
6.     if (y>0)
7.         y=y-1;
8.     z=z+x+y;
9. }
10. output (z);
```

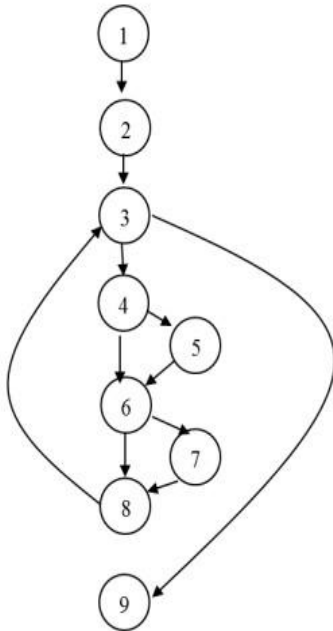
1. رسم مخطط الـ CFG للـ Code .
2. إيجاد testing paths for statement coverage .
3. إيجاد testing paths for branch coverage .
4. إيجاد testing paths for path coverage .
5. إيجاد test case لكل الـ path .

الحل:

1. لرسم الغراف نلاحظ أن العقدة 1 تمر على التسلسل مع 2، و 2 تمر على التسلسل مع 3 وتبدأ عملية التفرع عند العقدة 3. نعلم أن الحلقة عبارة عن repeated condition وبالتالي لدينا فرع في حال لم يتحقق الشرط يذهب إلى 9 ويطلع الخرج، أما في حال التحقق سيدخل إلى الحلقة. لدينا ضمن الحلقة شرطين الأول عند العقدة 4 والثاني عند العقدة 6. في حال تحقق الشرط عند العقدة 4 سيذهب إلى 5 وإذا لم يتحقق سيذهب إلى 6، وفي حال تحقق الشرط عند العقدة 6 سيذهب إلى 7 وإذا لم يتحقق سيذهب إلى 8. في العقدة 8 سنعود إلى العقدة 3 للتحقق من شرط الحلقة مرة أخرى.

2. نجد أن مساراً واحداً غطى كل العقد $statement = 9$

تذكرة: Test Case(input, expected output)



Testing Path (1): 1234567839 => test case (1,1,0)

أي أننا سنمر في هذا ال Path إذا كان الدخل $x = 1$, $y = 1$ وسيكون الخرج (وهو في مثالنا المتحول z) قيمته 0

3. نحتاج إلى ثلاث مسارات لتغطية كل الفروع: $Branches = 3$

(عند العقدة 3 و 4 و 6)

Testing Path (3): 1239 => test case (1, -1,0)

1234567839 => test case (1,1,0)

12346839

يمكننا نظرياً بمسارين أن نحصل على branch coverage

4. بما أنه يوجد لدينا حلقة نستطيع تجريب عدد لا نهائي من الحالات وبالتالي سنواجه مشكلة path explosion أي انفجار بعدد المسارات أي لا نستطيع تحقيق path coverage.

5. إيجاد test case لكل path

Path = لا نهائي

testing path = 5 (عدد المسارات التي يمكن اختبارها بغض النظر عن الحلقة)

1239 → test case (0,0,0)

1234567839 → test case (1,1,0)

123467839 → test case (0,1,0)

123456839 → test case (1,0,0)

12346839

عند إيجاد الـ test cases نلاحظ وجود مسار لا يمكن تحقيقه، حيث أن الذي يحكم المسارات هو الشروط والحلقات، وبالتالي لاختيار المدخلات ننظر لشروط دخول الحلقة والخروج منها.

للدخول إلى الحلقة يجب أن يكون مجموع $x + y > 0$ فالمسار 12346839 غير محقق حيث سيقابل كون x ضمن الشرط في العقدة 4 أصغر أو يساوي الصفر و y ضمن الشرط في العقدة 6 أصغر أو يساوي الصفر، وبالتالي مجموعهما أصغر أو يساوي الصفر، أي لم نستطيع دخول الحلقة من البداية، حيث لدخولها يجب أن يكون أحد الشرطين في 4 أو 6 محقق فقط و بالتالي هذا مسار مستحيل و لن يتحقق أبداً.